

Chapter 40

Time Series &

Forecasting

Break a time series into trend, seasonality, cycle, and noise · test
for stationarity and know why it matters · build the baselines
every forecast must beat, by hand

Approved chapter · v1

Contents

Chapter 40. Time Series & Forecasting

Why this matters

In plain English

40.1 What a time series is made of

40.2 Stationarity

40.3 Baselines that are hard to beat

40.4 Exponential smoothing

40.5 ARIMA and SARIMA

40.6 Machine learning for forecasting

40.7 Backtesting

40.8 Forecast accuracy: MAPE, WAPE, MASE

40.9 Demand forecasting for a manufacturer

40.10 Anomaly detection in sensor data

Common mistakes and how to spot them

In the real world: the planner who already had a model

Tools

The project: forecast Riverstone's monthly demand by product category

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 40. Time Series & Forecasting

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: break a time series into trend, seasonality, cycle, and noise · test for stationarity and know why it matters · build the baselines every forecast must beat, by hand · run exponential smoothing and Holt–Winters, and see the smoothing step on paper · read ACF and PACF plots and fit a SARIMA model · forecast with a machine learning model on lag features without leaking the future · backtest a forecast the way it will be used, and read the spread of errors · measure accuracy with MAPE, WAPE, and MASE, and know which to quote · turn a forecast into a production plan with safety stock · find anomalies in sensor data, including the kind that point-by-point checks miss.

Before you start: Chapter 21 (means, standard deviations, autocorrelation as an idea), Chapter 36 (splits and leakage), Chapter 37 (gradient boosting), and Chapter 39 (accuracy metrics). Chapter 18's pandas date handling is used throughout.

Time needed: 10–14 hours over two weeks.

Tools: Python 3 with pandas, statsmodels, scikit-learn, and Prophet (all free).

Practice data: two new Riverstone datasets built by seeded generators. **Demand:** units shipped per product category, weekly (365 weeks) and monthly (84 months), January 2019 to December 2025. **Sensor readings:** one week of one-minute readings from one injection-moulding machine (10,080 rows; the same generator builds the 4.8-million-row version used in Chapter 48). Every number in this chapter was calculated, and every output shown is real.

Why this matters

Every business runs on forecasts, most of them made in a spreadsheet by someone who copies last year's number and adds 10%. That method is better than it sounds, and one of the lessons of this chapter is that beating it is harder than most people expect.

- The production planner needs next quarter's demand per category to buy resin and schedule machines. Too low and orders ship late; too high and the warehouse fills with unsold stock.
- Finance needs a revenue forecast with an honest range around it, not a single number that will be wrong.
- The maintenance team wants to know that a machine's heater is failing before the shift supervisor notices scrap.
- An interviewer will ask what makes time series different from other data, and "the rows are in order" is a beginning, not an answer.

Time series break the assumption that most of Part IV relied on: that rows are independent. Yesterday predicts today. That's what makes forecasting possible, and it's what makes every shortcut from Chapter 36 (random splits, cross-validation, features that peek forward) fail quietly. This chapter is about doing it properly, and about measuring whether the effort beat the spreadsheet.

In plain English

Think of a shopkeeper who has kept a diary of daily sales for seven years.

They know the shop is busier every year (**trend**), that October and November are the peak because of the festive season (**seasonality**), that there are good years and bad years that last a while (**cycles**), and that any single day is unpredictable (**noise**). Reading the diary to separate those four is **decomposition**.

Asked what next October will bring, the shopkeeper's first answer is "about what last October brought, plus a bit for growth". That's the **seasonal naive** forecast, and it's surprisingly good. A more careful answer weighs recent months more than old ones (**exponential smoothing**), or notices that a good month tends to follow a good month (**autocorrelation**, the idea behind **ARIMA**).

To know whether a method works, the shopkeeper doesn't check it on last week; they ask, "if I'd used this method at the start of each of the last five years, how wrong would I have been?" That's **backtesting**. And "how wrong" is measured in units short or over across the whole year, not by the worst single day: that's **WAPE**.

Finally, if the shop's fridge thermometer reads 3 degrees higher every hour, the shopkeeper wants to know before the milk goes off, even though no single reading looks alarming. That's **anomaly detection in sensor data**, and the slow kind is the hard kind.

40.1 What a time series is made of

The data

```
import logging
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
logging.getLogger("cmdstanpy").setLevel(logging.ERROR)

weekly = pd.read_csv("../demand/weekly_demand.csv", parse_dates=["week_start"])
monthly = pd.read_csv("../demand/monthly_demand.csv", parse_dates=["month"])
print(
    f"weekly: {len(weekly):,} rows, {weekly['week_start'].min().date()} "
    f"to {weekly['week_start'].max().date()}"
)
print(
    "weekly problems: missing units",
    weekly["units"].isna().sum(),
    "| duplicated weeks",
    weekly.duplicated(["category", "week_start"]).sum(),
)
weekly = weekly.drop_duplicates(["category", "week_start"])
weekly["units"] = weekly.groupby("category")["units"].transform(
    lambda u: u.interpolate()
)

kitchen = (
    monthly[monthly["category"] == "Kitchen"]
    .set_index("month")["units"]
    .astype(float)
    .asfreq("MS")
)
print(f"\nKitchen, monthly: {len(kitchen)} months")
print(kitchen.groupby(kitchen.index.year).sum().astype(int).to_string())
```

```
weekly: 1,461 rows, 2019-01-07 to 2025-12-29
weekly problems: missing units 1 | duplicated weeks 1
```

```
Kitchen, monthly: 84 months
```

month	
2019	656605
2020	611652
2021	728878
2022	814600
2023	932607
2024	962052
2025	1051402

Kitchen products are Riverstone's largest category, and the monthly series is the one this chapter works with most. Two things in the weekly file were planted to be found: one missing week and one duplicated week. Real demand exports have both, and a forecasting model fed a duplicated week will learn a spike that never happened.

Four components

A time series is usually described as the combination of:

- **Trend:** the long-run direction. Kitchen units grew from 657,000 in 2019 to 1,051,000 in 2025.
- **Seasonality:** a pattern that repeats with a fixed period. Riverstone's peak is October–November, ahead of Diwali, when hotels and retailers stock up.
- **Cycles:** slower swings with no fixed period, such as the economy. Harder to model, often ignored.
- **Noise** (the residual): whatever is left after the others.

Decomposition estimates each part. The multiplicative version treats seasonality as a *factor* ("October is 1.43 times an average month"), which suits data that grows, because the seasonal swing grows with it:

```
from statsmodels.tsa.seasonal import seasonal_decompose

parts = seasonal_decompose(kitchen, model="multiplicative", period=12)
seasonal_index = parts.seasonal[:12]
seasonal_index.index = seasonal_index.index.strftime("%b")
print("seasonal index (1.0 = an average month):")
print(seasonal_index.round(3).to_string())
print(
    f"\ntrend, first and last available: {parts.trend.dropna().iloc[0]:,.0f} "
    f"-> {parts.trend.dropna().iloc[-1]:,.0f}"
)
print(
    f"residual: typical size {parts.resid.dropna().std():.3f} (as a multiplier), "
    f"largest {parts.resid.dropna().abs().max():.3f} "
    f"at {parts.resid.dropna().abs().idxmax():%b %Y}"
)
```

```
seasonal index (1.0 = an average month):
month
Jan    0.951
Feb    0.886
Mar    0.958
Apr    0.845
May    0.849
Jun    0.820
Jul    0.950
Aug    0.965
Sep    1.022
Oct    1.434
Nov    1.300
Dec    1.021

trend, first and last available: 55,477 -> 86,938
residual: typical size 0.105 (as a multiplier), largest 1.249 at Feb 2020
```

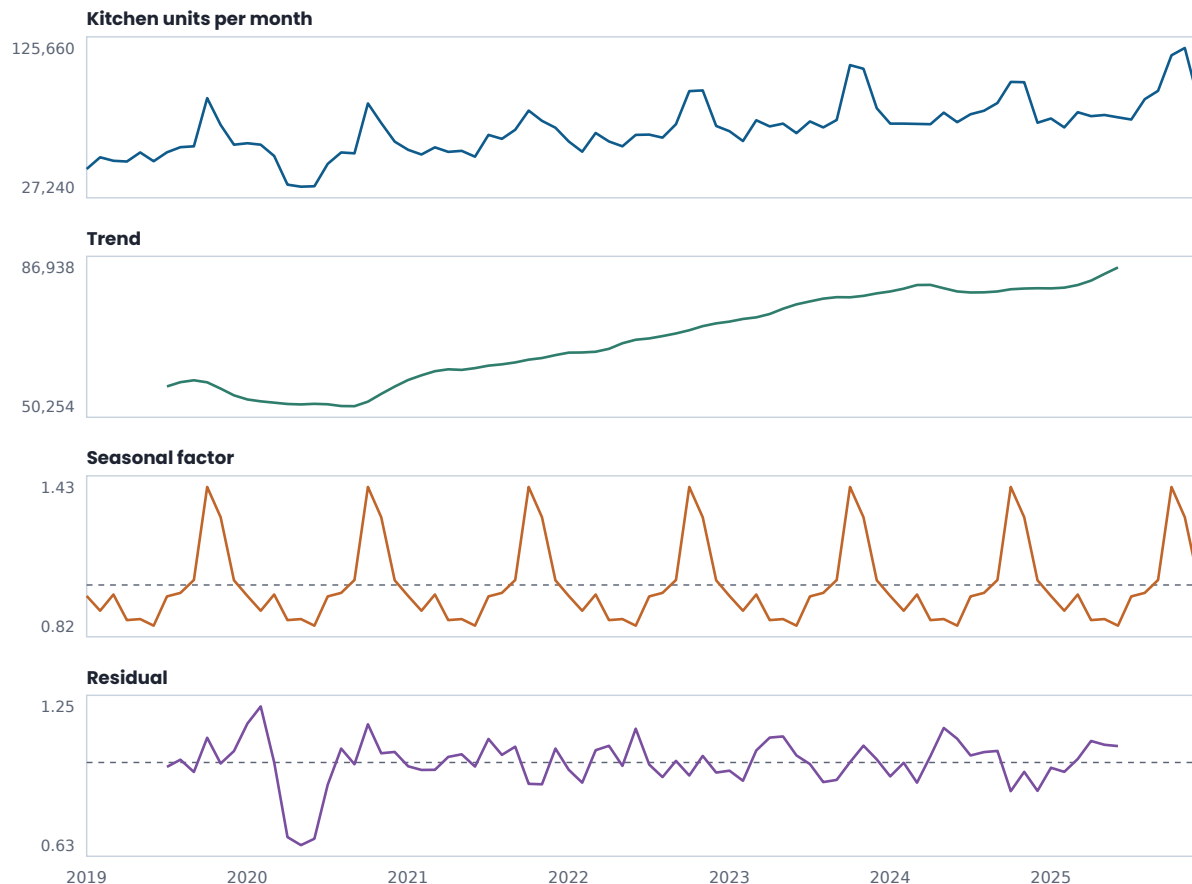


Figure 40.1 – Kitchen demand decomposed into trend, seasonality, and residual. The seasonal factor for October is 1.43; the residual's one large excursion is the spring of 2020.

Reading it. October runs at 143% of an average month and November at 130%, while April–June sit at about 82–85%. The trend roughly doubles over the period. The residual is typically about $\pm 10\%$, with one enormous exception: February to June 2020, when the factory shut and reopened. That event isn't seasonality or trend; it's a one-off that every model in this chapter has to live with.

Watch out: decomposition is a description, not a forecast. `seasonal_decompose` uses a centered moving average, which needs data on both sides of each point, so the trend has no value for the first and last six months. It's for understanding the series, not for predicting it.

40.2 Stationarity

Most classical forecasting methods assume the series is **stationary**: its mean, variance, and autocorrelation don't change over time. A series with a trend isn't stationary (the mean rises); one with growing seasonal swings isn't either (the variance rises).

The standard fix is **differencing**: model the *change* from one period to the next instead of the level. Seasonal differencing subtracts the value 12 months earlier. Taking logs first turns growing swings into steady ones. The **augmented Dickey–Fuller (ADF) test** checks the result; its null hypothesis is "not stationary", so a small p-value means stationary.

```
from statsmodels.tsa.stattools import adfuller

def adf_report(name, series):
    stat, p_value = adfuller(series.dropna())[:2]
    verdict = "stationary" if p_value < 0.05 else "NOT stationary"
    print(f"{name:<38} ADF statistic {stat:6.2f}    p-value {p_value:.4f}    {verdict}")

adf_report("units", kitchen)
adf_report("month-to-month change", kitchen.diff())
adf_report("log units, differenced", np.log(kitchen).diff())
adf_report("log units, differenced, then seasonal", np.log(kitchen).diff().diff(12))
```

units	ADF statistic	0.35	p-value 0.9797	NOT stationary
month-to-month change	ADF statistic	-3.95	p-value 0.0017	stationary
log units, differenced	ADF statistic	-3.52	p-value 0.0074	stationary
log units, differenced, then seasonal	ADF statistic	-3.55	p-value 0.0068	stationary

Reading it. The raw units fail ($p = 0.98$): of course, they trend upward. One difference is enough to pass, and the log-plus-seasonal-difference version, which section 40.5 uses, passes comfortably. The test is a check, not an oracle: it's weak on short series and says nothing about *which* model to fit. Its practical use is to tell you how many times to difference.

40.3 Baselines that are hard to beat

Chapter 36's rule applies with more force here: never report a forecast without the simple method it must beat. Three baselines, using 2019–2024 to forecast 2025:

- **Naive:** next month equals last month.
- **Seasonal naive:** next month equals the same month last year.
- **Moving average:** next month equals the average of the last 12.

```

train = kitchen[:"2024-12"]
test = kitchen["2025"]

def wape(actual, forecast):
    return (
        np.abs(np.asarray(actual) - np.asarray(forecast)).sum()
        / np.asarray(actual).sum()
    )

naive = np.repeat(train.iloc[-1], 12)
seasonal_naive = train["2024"].to_numpy()
moving_average = np.repeat(train.iloc[-12:].mean(), 12)
print(
    f"last value (Dec 2024): {train.iloc[-1]:,.0f}  "
    f" mean of 2024: {train.iloc[-12:].mean():,.0f}"
)
print(
    f"January 2025 actual {test.iloc[0]:,.0f}: naive says {naive[0]:,.0f}, "
    f"seasonal naive says {seasonal_naive[0]:,.0f} (Jan "
    f"2024), moving average says {moving_average[0]:,.0f}"
)
forecasts = {
    "naive": naive,
    "seasonal naive": seasonal_naive,
    "12-month moving average": moving_average,
}
for name, f in forecasts.items():
    print(f"{name:<26} WAPE over 2025 {wape(test, f):.1%}")

```

```

last value (Dec 2024): 72,627  mean of 2024: 80,171
January 2025 actual 75,641: naive says 72,627, seasonal naive says 71,958 (Jan 2024), moving
average says 80,171
naive                WAPE over 2025 17.7%
seasonal naive       WAPE over 2025 10.0%
12-month moving average WAPE over 2025 14.1%

```

How it works: WAPE, the accuracy measure used throughout, is the sum of absolute errors divided by the sum of actuals: "total units wrong as a share of total units". Section 40.8 works it by hand.

Reading it. The seasonal naive forecast, which needs no library and no skill, is within 10% over the year. Plain naive (17.7%) and the moving average (14.1%) do worse because they ignore the seasonal pattern entirely: the moving average predicts 80,171 for a January that runs well below average.

Seasonal naive is the number to beat for any strongly seasonal series, and several methods below don't beat it.

40.4 Exponential smoothing

Simple smoothing, by hand

Simple exponential smoothing keeps a running level that moves part of the way toward each new observation:

$$\text{new level} = \alpha \times \text{latest actual} + (1 - \alpha) \times \text{old level}$$

where α (alpha), between 0 and 1, sets how fast the level reacts. It's a weighted average of the whole history with weights that decay exponentially into the past. Five months of Kitchen data with $\alpha = 0.3$:

```
values = train["2024-08":"2024-12"]
alpha = 0.3
level = values.iloc[0]
print(f"start: level = first value = {level:,.0f}")
for month, y in values.iloc[1:].items():
    level = alpha * y + (1 - alpha) * level
    print(
        f"{month:%b %Y}: actual {y:>8,.0f}    level = 0.3 "
        f"x {y:,.0f} + 0.7 x previous = {level:>9,.0f}"
    )
```

```
start: level = first value = 81,128
Sep 2024: actual    86,644    level = 0.3 x 86,644 + 0.7 x previous =    82,783
Oct 2024: actual   101,587    level = 0.3 x 101,587 + 0.7 x previous =    88,424
Nov 2024: actual   101,361    level = 0.3 x 101,361 + 0.7 x previous =    92,305
Dec 2024: actual    72,627    level = 0.3 x 72,627 + 0.7 x previous =    86,402
```

Reading it. The level lags the actuals: it rises toward the October–November peak but never reaches it, and it comes down slowly in December. That's the point of smoothing (it ignores noise) and its weakness (it ignores real seasonal jumps too). Simple smoothing forecasts a flat line at the last level, so it can't follow seasonality.

Holt–Winters

Holt's method adds a second smoothed quantity for the trend (with its own parameter, β). **Holt–Winters** adds a third for the seasonal factors (γ). statsmodels chooses the parameters by minimizing the in-sample error:

```

from statsmodels.tsa.holtwinters import ExponentialSmoothing

ses = ExponentialSmoothing(train).fit()
holt_winters = ExponentialSmoothing(
    train, trend="add", seasonal="mul", seasonal_periods=12
).fit()
print(
    f"simple smoothing: alpha {ses.params['smoothing_level']:.3f}  "
    f"WAPE 2025 {wape(test, ses.forecast(12)):.1%}"
)
print(
    f"Holt-Winters:      alpha {holt_winters.params['smoothing_level']:.3f}  "
    f"beta {holt_winters.params['smoothing_trend']:.3f}  "
    f"gamma {holt_winters.params['smoothing_seasonal']:.3f}  "
    f"WAPE 2025 {wape(test, holt_winters.forecast(12)):.1%}"
)
forecasts["Holt-Winters"] = holt_winters.forecast(12).to_numpy()
hw_fc = holt_winters.forecast(12)
print("Holt-Winters vs actual, first four months of 2025:")
print(
    pd.DataFrame({"actual": test[:4], "forecast": hw_fc[:4].round()})
    .astype(int)
    .to_string()
)

```

```

simple smoothing: alpha 0.993  WAPE 2025 17.5%
Holt-Winters:      alpha 0.725  beta 0.011  gamma 0.000  WAPE 2025 17.5%
Holt-Winters vs actual, first four months of 2025:
      actual  forecast
2025-01-01  75641     67183
2025-02-01  69282     64400
2025-03-01  80091     68328
2025-04-01  77281     62052

```

Reading it, plainly. Holt–Winters scores **17.5%**, worse than seasonal naive's 10%. Look at why: the fitted α is 0.725, so the level chases the latest month, and γ is 0.000, so the seasonal factors were fixed from the early years and never updated. Starting from a low December 2024, the forecast for early 2025 runs 10–15% below actual. The optimizer minimized *in-sample* error and found a combination that fits the past and forecasts badly. This is a common outcome on short, noisy series; the fix is usually to constrain the parameters (`use_boxcox` , `damped_trend` , or fixing γ) and to judge by backtest, not by in-sample fit.

When smoothing works well: stable series with clear seasonality and lots of history, and as a fast, explainable default in planning tools (most commercial demand-planning software runs some form of Holt–Winters). It's also the method a planner can maintain in a spreadsheet.

40.5 ARIMA and SARIMA

The idea

ARIMA models a stationary series as a combination of:

- **AR** (autoregressive, order p): this month depends on the last p months' values.
- **I** (integrated, order d): the series was differenced d times to make it stationary.
- **MA** (moving average, order q): this month depends on the last q months' forecast errors.

SARIMA adds the same three at the seasonal lag (12 months here), written $(p, d, q)(P, D, Q)_s$. The most common seasonal model in practice, $(1, 1, 1)(0, 1, 1)_{12}$, says: difference once and seasonally once, then explain what's left with one autoregressive term, one error term, and one seasonal error term.

Reading ACF and PACF

The **autocorrelation function (ACF)** measures how a series correlates with itself at each lag; the **partial autocorrelation (PACF)** does the same after removing the effect of shorter lags. On a properly differenced series, spikes in the ACF suggest MA terms and spikes in the PACF suggest AR terms:

```
from statsmodels.tsa.stattools import acf, pacf

stationary = np.log(train).diff().diff(12).dropna()
lags = [1, 2, 3, 6, 11, 12, 13, 24]
acf_values = acf(stationary, nlags=24)
pacf_values = pacf(stationary, nlags=24)
print("lag   ACF       PACF       (differenced log series)")
for lag in lags:
    print(f"{lag:>3}   {acf_values[lag]:6.2f}   {pacf_values[lag]:6.2f}")
print(f"significance line: about +/-{2 / np.sqrt(len(stationary)):.2f}")
```

lag	ACF	PACF	(differenced log series)
1	0.06	0.06	
2	-0.07	-0.07	
3	-0.12	-0.11	
6	-0.06	-0.11	
11	-0.06	-0.02	
12	-0.34	-0.48	
13	-0.06	0.09	
24	-0.06	-0.05	

significance line: about +/-0.26

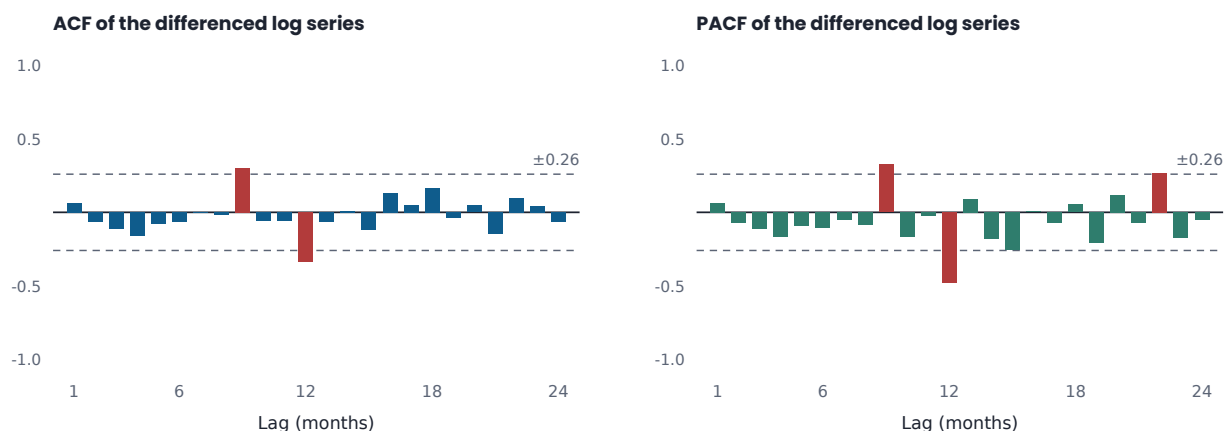


Figure 40.2 – ACF and PACF of the differenced log series. Almost everything sits inside the significance band except lag 12, the sign of a seasonal moving-average term.

Reading it. After differencing, almost nothing is left except a clear negative spike at lag 12 in both plots (−0.34 and −0.48, well outside the ± 0.26 band). A negative ACF spike at the seasonal lag after seasonal differencing is the textbook sign of a seasonal MA(1) term, which is the $Q = 1$ in the model above. The short lags are all within the band, so p and q are small. In practice most people try the standard model, check the residuals, and then let `auto_arima` (from the `pmdarima` package) or a small grid search over orders confirm it.

Fitting SARIMA

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima = SARIMAX(np.log(train), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(
    disp=False
)
print(sarima.summary().tables[1])
sarima_fc = np.exp(sarima.forecast(12))
forecasts["SARIMA"] = sarima_fc.to_numpy()
print(
    f"\nSARIMA (1,1,1)(0,1,1)12 on log units: WAPE 2025 "
    f"{waape(test, sarima_fc):.1%}    AIC {sarima.aic:.1f}"
)
interval = np.exp(sarima.get_forecast(12).conf_int(alpha=0.2))
print(
    f"December 2025: forecast {sarima_fc.iloc[-1]:.0f}, 80% interval "
    f"{interval.iloc[-1, 0]:.0f} to "
    f"{interval.iloc[-1, 1]:.0f}, actual {test.iloc[-1]:.0f}"
)
```

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.6969        0.175        3.972      0.000         0.353         1.041
ma.L1         -0.9974        4.852       -0.206      0.837        -10.507         8.512
ma.S.L12      -0.6357        0.236       -2.693      0.007         -1.098        -0.173
sigma2         0.0162        0.077         0.211      0.833         -0.135         0.167
=====
```

```
SARIMA (1,1,1)(0,1,1)12 on log units: WAPE 2025 4.0%    AIC -58.0
December 2025: forecast 90,680, 80% interval 70,932 to 115,926, actual 88,926
```

How it works: the model is fitted to **log** units so that the seasonal swing is proportional and the forecast can't go negative; `np.exp` converts back. `get_forecast(...).conf_int(alpha=0.2)` gives an 80% prediction interval.

Reading it. SARIMA scores **4.0%** on 2025, far better than seasonal naive. The coefficients say what the plots said: a seasonal MA term of −0.64 (significant) and an AR(1) of 0.70. The MA(1) coefficient of −0.997 with a huge standard error looks like a redundant term; exercise 7 tests whether it can be dropped, and the answer is a clear no. A coefficient near the edge of its allowed range inflates its standard error without making the term useless. The December forecast of 90,680 came with an 80% interval of 71,000 to 116,000; the actual, 88,926, landed inside it. **Always show the interval.** A point forecast without one invites the reader to believe it.

Forecasting libraries

Beyond statsmodels, three tools are worth knowing:

- **Prophet** (Meta): fits trend, seasonality, and holiday effects as additive curves, handles missing data and outliers gracefully, and needs almost no tuning. Popular with analysts for exactly that reason.
- **pmdarima**: `auto_arima`, which searches SARIMA orders automatically.
- **statsforecast** and **sktime**: fast, scikit-learn-style libraries for forecasting many series at once, which is what a demand planner with 500 products needs.

```
from prophet import Prophet

frame = pd.DataFrame({"ds": train.index, "y": train.to_numpy()})
prophet = Prophet(
    yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
)
prophet.fit(frame)
prophet_fc = prophet.predict(pd.DataFrame({"ds": test.index}))["yhat"].to_numpy()
forecasts["Prophet"] = prophet_fc
print(f"Prophet: WAPE 2025 {wape(test, prophet_fc):.1%}")
```

```
Prophet: WAPE 2025 5.9%
```

Reading it. Prophet scores **5.9%** on 2025 without any settings beyond "there's yearly seasonality". Not as good as the hand-chosen SARIMA here, and much better than Holt-Winters or seasonal naive, for one line of configuration. For someone forecasting dozens of series with no time to tune each, that trade is often the right one.

40.6 Machine learning for forecasting

Gradient boosting (Chapter 37) can forecast too, if the time series is turned into a table: each row is one period, and its features are **lags** (the values 1, 2, ..., 52 periods earlier), **rolling statistics** of past values, and **calendar** features (week of year). The target is this period's value. With 84 months that's too few rows; the weekly series (365 rows) is where this approach starts to make sense.

```

from sklearn.ensemble import HistGradientBoostingRegressor

wk = (
    weekly[weekly["category"] == "Kitchen"]
    .set_index("week_start")["units"]
    .astype(float)
)
features = pd.DataFrame({"units": wk})
for lag in [1, 2, 3, 4, 8, 13, 26, 52]:
    features[f"lag_{lag}"] = wk.shift(lag)
features["mean_4"] = wk.shift(1).rolling(4).mean()
features["mean_13"] = wk.shift(1).rolling(13).mean()
features["week_of_year"] = features.index.isocalendar().week.to_numpy().astype(int)
features["year"] = features.index.year
features = features.dropna()
wk_train, wk_test = features.loc[:"2024-12-31"], features.loc["2025"]
X_cols = [c for c in features.columns if c != "units"]
gbr = HistGradientBoostingRegressor(
    max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
)
gbr.fit(wk_train[X_cols], wk_train["units"])
print(
    f"{len(wk_train)} training weeks, {len(wk_test)} test weeks, {len(X_cols)} features"
)
print(
    f"one-week-ahead WAPE on 2025 weeks: "
    f"{wape(wk_test['units'], gbr.predict(wk_test[X_cols])):.1%}"
)
wk_seasonal_naive = wk.shift(52).reindex(wk_test.index)
print(
    f"weekly seasonal naive (same week last year): "
    f"{wape(wk_test['units'], wk_seasonal_naive):.1%}"
)

```

```

261 training weeks, 52 test weeks, 12 features
one-week-ahead WAPE on 2025 weeks: 9.3%
weekly seasonal naive (same week last year): 11.3%

```

How it works:

- Every feature is built from `wk.shift(...)`: values strictly *before* the row's week.
`rolling(4).mean()` is applied to the **shifted** series, so the 4-week average never includes the current week. That's the time-series version of Chapter 36's prediction-moment rule, and getting it wrong (using `rolling(4)` on the unshifted series) is the most common leak in ML forecasting.
- The split is by time: train to December 2024, test on 2025.

Reading it. One week ahead, the model's WAPE is 9.3% against 11.3% for the weekly seasonal naive. But "one week ahead" means each prediction used the *actual* previous weeks, which a planner forecasting a year out won't have. The fair test is **recursive**: forecast week 1, feed the forecast in as the lag for week 2, and so on:

```

history = wk[:"2024-12-31"].copy()
recursive = []
for week in wk_test.index:
    row = {f"lag_{lag}": history.iloc[-lag] for lag in [1, 2, 3, 4, 8, 13, 26, 52]}
    row["mean_4"] = history.iloc[-4:].mean()
    row["mean_13"] = history.iloc[-13:].mean()
    row["week_of_year"] = int(week.isocalendar().week)
    row["year"] = week.year
    prediction = gbr.predict(pd.DataFrame([row])[X_cols])[0]
    recursive.append(prediction)
    history.loc[week] = prediction # feed the forecast back in, never the actual
recursive = pd.Series(recursive, index=wk_test.index)
print(
    f"52-week recursive forecast, WAPE on 2025: {wape(wk_test['units'], recursive):.1%}"
)
monthly_from_weekly = recursive.resample("MS").sum()
print(
    f"summed to months and compared with the monthly series: "
    f"{wape(test, monthly_from_weekly.reindex(test.index)):.1%}"
)

```

```

52-week recursive forecast, WAPE on 2025: 8.9%
summed to months and compared with the monthly series: 13.6%

```

Reading it. Recursively, the weekly model scores 8.9% over 2025, still ahead of the weekly baseline. Summed into months it scores 13.6% against the monthly series, worse than SARIMA's 4.0%, partly because weekly noise is harder to forecast than monthly totals and errors compound over 52 recursive steps. That's the honest position of ML forecasting on a single clean series: competitive, not dominant. Where it wins is with **many series and external features**: promotions, prices, weather, holidays, and the demand of related products, which SARIMA can't easily use. Chapter 48's sensor work and the capstone use it that way.

Watch out: the calendar can leak too. A feature such as "total units this year" or "average price this quarter" includes the future if computed over the whole period. Every feature must be computable on the forecast date from data available then.

40.7 Backtesting

One test year is one sample. 2025 happened to be an ordinary year; a method that does well on it might have failed in 2020. **Backtesting** (rolling-origin evaluation) repeats the forecast from several past starting points and looks at all the scores:

```
def backtest(series, make_forecast, horizon=12, folds=5):
    scores = []
    for fold in range(folds):
        cut = len(series) - horizon * (folds - fold)
        history, future = series.iloc[:cut], series.iloc[cut : cut + horizon]
        scores.append(wape(future, make_forecast(history, horizon)))
    return np.array(scores)

methods = {
    "seasonal naive": lambda h, n: h.iloc[-12:].to_numpy()[n],
    "Holt-Winters": lambda h, n: ExponentialSmoothing(
        h, trend="add", seasonal="mul", seasonal_periods=12
    )
    .fit()
    .forecast(n)
    .to_numpy(),
    "SARIMA": lambda h, n: np.exp(
        SARIMAX(np.log(h), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12))
        .fit(dispatch=False)
        .forecast(n)
    ).to_numpy(),
}
print("backtest: 5 folds, each forecasting the next 12 months from 2021 to 2025")
print(
    f"{'method':<16} "
    + " ".join(f"fold {i + 1}" for i in range(5))
    + "    mean    worst"
)
for name, fn in methods.items():
    s = backtest(kitchen, fn)
    print(
        f"{'name':<16} "
        + " ".join(f"{v:6.1%}" for v in s)
        + f"    {s.mean():6.1%}    {s.max():6.1%}"
    )
```

```
backtest: 5 folds, each forecasting the next 12 months from 2021 to 2025
method      fold 1  fold 2  fold 3  fold 4  fold 5    mean    worst
seasonal naive  20.7%  10.5%  12.7%  10.2%  10.0%   12.8%   20.7%
Holt-Winters   14.4%  11.9%  14.0%   8.2%  17.5%   13.2%   17.5%
SARIMA        16.6%  15.0%  10.8%   9.9%   4.0%   11.3%   16.6%
```

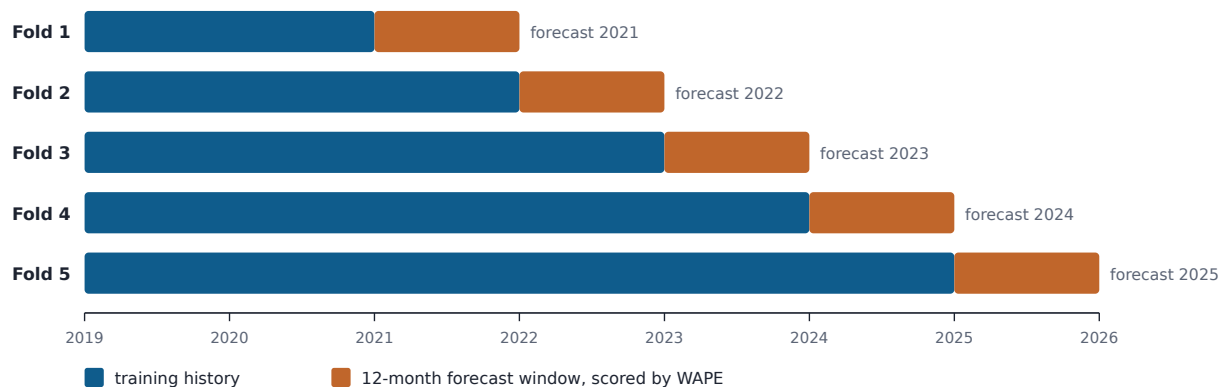


Figure 40.3 – Rolling-origin backtesting. Each fold trains on everything before its cut and forecasts the next 12 months, so every method is judged on five different years.

Reading it, plainly. On a single year (fold 5, which is 2025), SARIMA's 4.0% looks like a clear win. Over five years the picture is more modest: SARIMA averages **11.3%**, seasonal naive 12.8%, and Holt–Winters 13.2%. The worst years are 2021 for seasonal naive (20.7%, because it copied the shut-down months of 2020) and 2021 for SARIMA too (16.6%, for the related reason that it was trained through the shutdown). **The 2025 result overstated SARIMA's advantage by a factor of three.** This is why a single test period is never enough for a forecast: report the backtest mean and the worst fold, and expect the worst fold to happen again.

Which fold to trust? All of them, but not equally. The most recent folds are closest to how the model will be used; the 2020–2021 folds show how each method copes with a shock. A planner would rightly ask, "what happens next time something like 2020 happens?", and only the backtest can answer.

40.8 Forecast accuracy: MAPE, WAPE, MASE

Three measures, all on SARIMA's 2025 forecast, worked from the twelve monthly errors:

- **MAPE** (mean absolute percentage error): average of $|\text{error}| \div \text{actual}$ across months. Familiar, and flawed: it treats a 10% miss in a small month the same as in a big one, it can't handle zero actuals, and it penalizes over-forecasting more than under-forecasting.
- **WAPE** (weighted absolute percentage error): sum of $|\text{errors}| \div \text{sum of actuals}$. Big months count more, which is what a planner wants; it's stable with zeros; it equals "total units wrong $\div$ total units". **This is the number to quote.**
- **MASE** (mean absolute scaled error): mean $|\text{error}| \div$ the mean absolute error of the seasonal naive method on the training data. Below 1 means the model beats seasonal naive *on the scale of the training history*, and it's comparable across series of different sizes.

```
actual = test.to_numpy()
forecast = forecasts["SARIMA"]
errors = actual - forecast
mape = np.mean(np.abs(errors) / actual)
wape_value = np.abs(errors).sum() / actual.sum()
naive_in_sample = np.abs(train.to_numpy()[12:] - train.to_numpy()[:-12]).mean()
mase = np.abs(errors).mean() / naive_in_sample
print(
    f"MAPE {mape:.1%}    WAPE {wape_value:.1%}    MASE "
    f"{mase:.2f}    bias {errors.mean():+.0f} units/month"
)
print(
    f"largest month: {np.abs(errors).max():+.0f} units "
    f"({test.index[np.abs(errors).argmax()]:%b %Y})"
)
print(
    f"total 2025 actual {actual.sum():+.0f}, total forecast {forecast.sum():+.0f} "
    f"({(forecast.sum() - actual.sum()) / actual.sum():+.1%})"
)
```

```
MAPE 4.1%    WAPE 4.0%    MASE 0.37    bias +776 units/month
largest month: 10,464 units (Jul 2025)
total 2025 actual 1,051,402, total forecast 1,042,096 (-0.9%)
```

By hand for one month: in July 2025 the forecast missed the actual 74,926 by 10,464 units, 14%. Over the year the absolute errors add to about 42,000 against 1,051,402 actual units: $WAPE = 42,000 \div 1,051,402 = 4.0\%$. MAPE is slightly higher (4.1%) because it weights the small months' percentages equally. MASE of 0.37 says the model's average error is about a third of what seasonal naive achieved on the training data.

Bias matters too: the errors average +776 units a month, meaning the forecast ran slightly low, and the annual total was 0.9% under. A forecast that's 4% wrong month to month but 1% wrong for the year is fine for resin purchasing and less fine for weekly scheduling. Say which horizon the number refers to.

40.9 Demand forecasting for a manufacturer

All four categories

The project asks for a forecast by category. Here's the fair comparison, four methods against seasonal naive, on 2025:

```

rows = []
for cat in ["Storage", "Kitchen", "Industrial", "Furniture"]:
    series = (
        monthly[monthly["category"] == cat]
        .set_index("month")["units"]
        .astype(float)
        .asfreq("MS")
    )
    tr, te = series[:"2024-12"], series["2025"]
    hw_fc = (
        ExponentialSmoothing(tr, trend="add", seasonal="mul", seasonal_periods=12)
        .fit()
        .forecast(12)
    )
    sar_fc = np.exp(
        SARIMAX(np.log(tr), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12))
        .fit(dispatch=False)
        .forecast(12)
    )
    pr = Prophet(
        yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
    )
    pr.fit(pd.DataFrame({"ds": tr.index, "y": tr.to_numpy()}))
    pr_fc = pr.predict(pd.DataFrame({"ds": te.index}))["yhat"].to_numpy()
    rows.append(
        {
            "category": cat,
            "2025 units": int(te.sum()),
            "seasonal naive": wape(te, tr["2024"].to_numpy()),
            "Holt-Winters": wape(te, hw_fc),
            "SARIMA": wape(te, sar_fc),
            "Prophet": wape(te, pr_fc),
        }
    )
table = pd.DataFrame(rows).set_index("category")
print(
    (table.drop(columns="2025 units") * 100)
    .round(1)
    .astype(str)
    .add("%")
    .assign(units=table["2025 units"].map("{:,}".format))
    .to_string()
)

```

	seasonal naive	Holt-Winters	SARIMA	Prophet	units
category					
Storage	7.7%	7.2%	3.7%	6.7%	691,712
Kitchen	10.0%	17.5%	4.0%	5.9%	1,051,402
Industrial	5.7%	6.7%	6.7%	7.8%	400,694
Furniture	15.8%	9.3%	7.4%	9.6%	411,884

Reading it. No method wins everywhere. SARIMA is best on three categories; on Industrial, the least seasonal series, seasonal naive (5.7%) beats every model, and SARIMA and Holt-Winters both score worse than copying last year. Holt-Winters is fine on Storage and Furniture and poor on Kitchen. Prophet is never best and never bad on 2025 (its backtest, exercise 9, is less flattering). A planner choosing one method for all four would pick SARIMA; a planner choosing per category, judged by backtest rather than by 2025 alone, would run seasonal naive for Industrial and keep the model for the rest. **The comparison table is the deliverable**, more than any forecast.

From forecast to plan

A forecast becomes a production plan when it's combined with an allowance for being wrong. The simplest version: plan for the forecast plus **safety stock** sized from the backtest error:

```
sarima_scores = backtest(kitchen, methods["SARIMA"])
error_allowance = sarima_scores.mean() + sarima_scores.std()
plan = pd.DataFrame(
    {
        "forecast": sarima_fc.round(),
        "safety stock": (sarima_fc * error_allowance).round(),
    }
)
plan["plan"] = plan["forecast"] + plan["safety stock"]
plan["actual"] = test
plan.index = plan.index.strftime("%b %Y")
print(
    f"error allowance from the backtest: mean WAPE {sarima_scores.mean():.1%} "
    f"+ one sd {sarima_scores.std():.1%} = {error_allowance:.1%}"
)
print(plan.head(6).astype(int).to_string())
print(
    f"months where actual exceeded the plan: {(plan['actual'] > plan['plan']).sum()} of 12"
)
```

```
error allowance from the backtest: mean WAPE 11.3% + one sd 4.4% = 15.7%
   forecast  safety stock   plan  actual
Jan 2025    69547        10890  80437   75641
Feb 2025    69554        10891  80445   69282
Mar 2025    77752        12175  89927   80091
Apr 2025    73463        11503  84966   77281
May 2025    77806        12183  89989   78092
Jun 2025    74725        11701  86426   76488
months where actual exceeded the plan: 0 of 12
```

Reading it. The backtest says SARIMA is typically 11% off with a spread of 4%, so the plan carries 15.7% above forecast. In 2025 that covered every month; in a year like 2021 it wouldn't have. That's the trade the planner makes explicitly: more safety stock costs warehouse space and cash, less risks late orders, and the backtest is what makes the trade a calculation instead of a guess. Chapter 45's business metrics put rupee values on both sides.

What to tell the production planner. "For Kitchen, plan on the model's monthly forecast plus 16%, which would have covered every month this year. For Industrial, last year's numbers are as good as any model. October and November need 40% more capacity than an average month, and the model can't see a shock like 2020 coming, so keep the safety stock through the festive season."

40.10 Anomaly detection in sensor data

A week of readings

Riverstone's moulding machines log temperature, pressure, and cycle time every minute. The maintenance question isn't "forecast the temperature"; it's "tell me when something is wrong". Three kinds of wrong were planted in this week: a heater fault (temperature ramps up over 50 minutes), a

pressure spike, and a stoppage.

```
readings = pd.read_csv(
    "../sensors/machine_readings_week.csv", parse_dates=["timestamp"]
)
print(
    f"{len(readings):,} one-minute readings from {readings['machine_id'].iloc[0]}, "
    f"{readings['timestamp'].min()} to {readings['timestamp'].max()}"
)
print(
    readings[["temperature_c", "pressure_bar", "cycle_seconds"]]
    .describe()
    .loc[["mean", "std", "min", "max"]]
    .round(2)
    .to_string()
)
print("status counts:", readings["status"].value_counts().to_dict())
print("missing temperature readings:", readings["temperature_c"].isna().sum())
```

```
10,080 one-minute readings from M01, 2025-03-03 00:00:00 to 2025-03-09 23:59:00
      temperature_c  pressure_bar  cycle_seconds
mean           215.35         117.66           18.50
std              2.53           6.84            0.26
min            209.33           0.00           17.56
max            225.25          150.98           19.61
status counts: {'running': 10048, 'stopped': 32}
missing temperature readings: 18
```

Point anomalies: the rolling z-score

The obvious first tool: compare each reading with the trailing two hours, and flag anything more than four standard deviations away.

```
temp = readings.set_index("timestamp")["temperature_c"].interpolate()
rolling_mean = (
    temp.rolling("120min").mean().shift(1)
) # the past two hours, not including now
rolling_std = temp.rolling("120min").std().shift(1)
z = (temp - rolling_mean) / rolling_std
alerts = z[z.abs() > 4]
print(
    f"readings more than 4 standard deviations from the trailing 2-hour mean: {len(alerts)}"
)
for when, value in alerts.items():
    print(
        f"   {when:%a %H:%M}: {temp[when]:.1f} C against a trailing "
        f"mean of {rolling_mean[when]:.1f} (z = {value:+.1f})"
    )
hottest = temp.idxmax()
print(
    f"\nhottest minute of the week: {hottest:%a %H:%M} at {temp[hottest]:.1f} C, "
    f"z-score there only {z[hottest]:+.1f}"
)
```

```
readings more than 4 standard deviations from the trailing 2-hour mean: 3
Sat 18:49: 214.2 C against a trailing mean of 217.8 (z = -4.3)
Sun 07:29: 216.8 C against a trailing mean of 213.0 (z = +4.9)
Sun 11:29: 220.0 C against a trailing mean of 216.7 (z = +4.1)

hottest minute of the week: Wed 22:53 at 225.2 C, z-score there only +2.8
```

How it works: `rolling("120min")` uses a time-based window; `.shift(1)` excludes the current reading from its own baseline, so a spike can't hide itself. The missing readings are interpolated first.

Reading it. The z-score flags three single readings, each a one-minute blip of 3–4 degrees that reverts immediately: noise, not faults. And it **missed the heater fault**. The hottest minute of the week, 225.2 °C on Wednesday night, had a z-score of only +2.8. A ramp that rises a fraction of a degree a minute drags the trailing mean and standard deviation up with it, so no single reading ever looks far from "normal". **Point-by-point tests can't see slow drift.** That's the most important thing to know about sensor monitoring.

Slow drift: a slow baseline and persistence

Two changes fix it: measure deviation from a **slow, robust baseline** (the trailing 24-hour median, which a 50-minute ramp can't drag), and require the deviation to **persist** for several readings before alerting, which silences the blips:

```

baseline = (
    temp.rolling("24h").median().shift(1)
) # a slow, robust "normal" for the machine
excess = temp - baseline
sustained = (excess > 6).rolling(
    10
).sum() == 10 # more than 6 C above normal for 10 minutes running
first = sustained[sustained].index.min()
episode = excess[(excess > 6) & (excess.index >= first - pd.Timedelta("15min"))]
print(
    f"heater fault: temperature first exceeded normal by "
    f"6 C for 10 straight minutes at {first:%a %H:%M}"
)
print(
    f" episode from {episode.index.min():%H:%M} to "
    f"{episode.index.max():%H:%M}, peak excess {excess.max():.1f} C"
)

pressure = readings.set_index("timestamp")["pressure_bar"]
running = readings.set_index("timestamp")["status"] == "running"
pressure = pressure.where(running) # ignore minutes when the machine was stopped
smooth = (
    pressure.ewm(halflife="30min", times=pressure.index, ignore_na=True).mean().shift(1)
)
residual = (pressure - smooth).dropna()
threshold = 5 * residual[:"2025-03-03 23:59"].std()
spikes = residual[residual.abs() > threshold]
print(
    f"pressure spikes above {threshold:.1f} bar (5 sd of day-one residuals): "
    f"{len(spikes)} readings at {[t.strftime('%a %H:%M') for t in spikes.index]}"
)
stops = readings[readings["status"] == "stopped"]
print(
    f"stoppage: {stops['timestamp'].min():%a %H:%M} to "
    f"{stops['timestamp'].max():%H:%M} ({len(stops)} minutes)"
)

```

```

heater fault: temperature first exceeded normal by 6 C for 10 straight minutes at Wed 22:51
 episode from 22:37 to 23:04, peak excess 9.7 C
pressure spikes above 7.5 bar (5 sd of day-one residuals): 3 readings at ['Tue 12:00', 'Tue 12:01',
'Tue 12:02']
stoppage: Wed 22:02 to 22:33 (32 minutes)

```

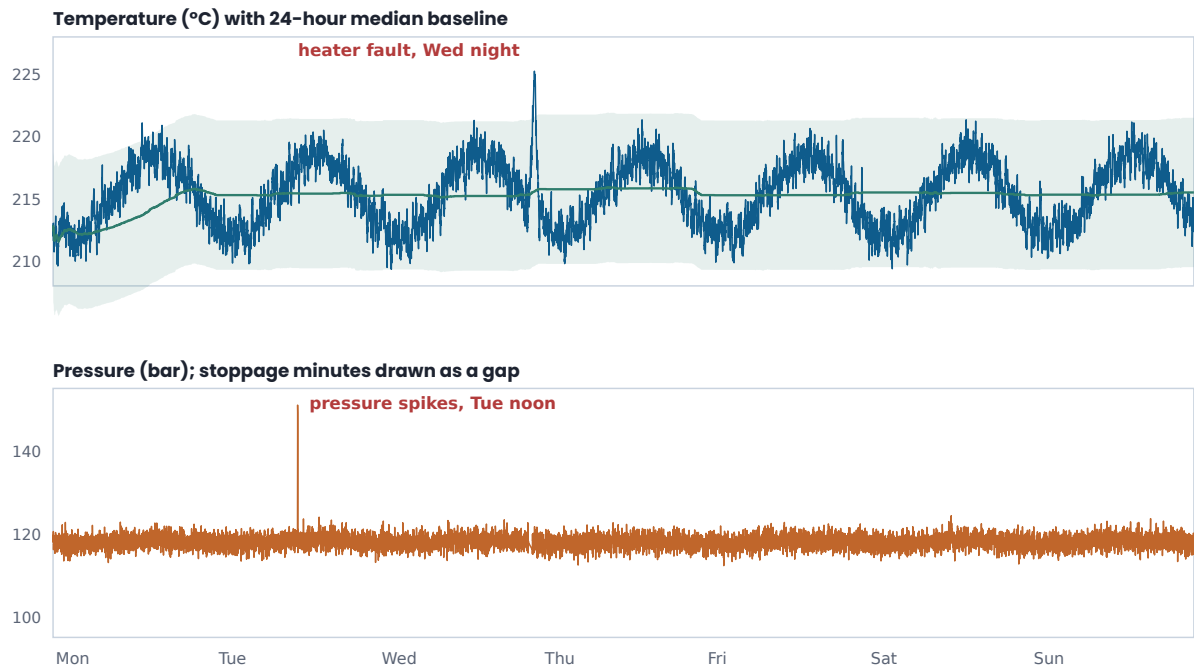


Figure 40.4 — Sensor anomalies. Top: the heater fault stands out against a 24-hour median baseline, though no single minute was extreme. Bottom: pressure spikes caught by the forecast-residual rule, and the stoppage, which is a status change rather than a sensor anomaly.

Reading it. The persistence rule fires at 22:51 on Wednesday. The temperature had crossed the 6 °C line at 22:37, minutes after the machine restarted from its 22:02–22:33 stoppage (a link an engineer would want pointed out), and stayed high until 23:04. The alert comes late in the ramp; exercise 13 shows how a lower threshold trades earlier warning for false alerts. The z-score, by contrast, never fired at all. The pressure rule uses a **forecast residual**: an exponentially weighted forecast of "what pressure should be now", with anything more than 5 standard deviations off flagged. It finds exactly the three planted readings at Tuesday noon and nothing else, once the stoppage minutes (pressure zero, and not an anomaly) are excluded by status.

Three general rules for sensor anomalies: **use the status column** (a stopped machine isn't an anomaly), **choose the baseline for the fault you're looking for** (fast for spikes, slow for drift), and **require persistence** before paging anyone. Chapter 48 runs this on 12 machines and 40 weeks, where the question becomes one of engineering rather than statistics.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Random train/test split on a time series	Wonderful validation scores, bad forecasts	Split by time; backtest with rolling origins
No seasonal naive baseline	"Our model is 12% accurate" with no comparison	Always report seasonal naive alongside
Judging on one test period	Method chosen on a lucky year	Backtest over several years; report mean and worst fold

Mistake	Symptom	Fix
Rolling features that include the current period	ML forecast looks too good one step ahead	<code>shift(1)</code> before <code>rolling(...)</code>
One-step-ahead scores for a multi-step problem	Model can't deliver in practice	Evaluate recursively or with direct multi-horizon models
Trusting in-sample fit	Holt-Winters with $\gamma = 0$ and a bad forecast	Choose parameters by backtest
Forecasting raw units with growing seasonality	Intervals too narrow at peaks; negative forecasts	Log-transform or use a multiplicative model
Quoting MAPE on lumpy or small series	Huge percentages from tiny months	Use WAPE; add MASE for comparisons
A point forecast with no interval	Readers believe the number	Report an 80% or 95% prediction interval
Duplicated or missing periods in the input	Phantom spikes; models that refuse to fit	Check for duplicates and gaps before anything else
Forecasting through a known one-off	The 2020 dip repeats in the model's mind	Dummy variable, or exclude the period, and say so
Ignoring the status column in sensor data	Stoppages flagged as anomalies	Mask by status first
Point z-scores for slow drift	Heater fault missed entirely	Slow robust baseline plus a persistence rule
Alerting on single readings	Maintenance ignores the alerts	Require n consecutive readings

In the real world: the planner who already had a model

In July 2026, Riverstone's production planner, Deepak Nair, shows Meera his demand sheet. It has a column called "model": last year's month times a growth factor he adjusts by hand. He's been using it for four years, and he asks, slightly defensively, whether "the data science" is going to replace it.

Meera doesn't answer until she's backtested his sheet against everything in this chapter. His method is seasonal naive with a hand-tuned growth factor. Over five years, it scores a mean WAPE of 11.9% on Kitchen, better than Holt-Winters and within a point of SARIMA. On Industrial it beats every model.

She shows him the table. Then she shows him two things his sheet can't do: an **interval** (his single number gives the resin buyer no idea how much to hedge), and the **2021 fold**, where his sheet, like seasonal naive, copied the shutdown and under-forecast the recovery by 20%.

The result isn't a replacement. Deepak keeps his sheet for Industrial and as the sanity check for everything else. The SARIMA forecasts go into a second column, with an 80% interval in a third, and safety stock is now sized from the backtest error instead of "15% because it's always been 15%". Once a quarter, Meera re-runs the backtest and they look together at whether the model is still earning its column.

The thing Deepak says afterwards is the thing to remember: *"So the model's job is to tell me how wrong I'm likely to be."* That's most of forecasting.

Tools

- **pandas** (3.0.2): `date indexes`, `asfreq`, `shift`, `rolling` with time-based windows, `resample`, `ewm` with `times`.
 - **statsmodels** 0.15.0: `seasonal_decompose`, `adfuller`, `acf`, `pacf`, `ExponentialSmoothing`, `SARIMAX`. Its documentation on state-space models is the reference for prediction intervals.
 - **Prophet** 1.4.0 (`pip install prophet`; installs CmdStan on first use, which can take a few minutes).
 - **scikit-learn** 1.8.0 for `HistGradientBoostingRegressor`.
 - Not used here but worth knowing: **pmdarima** (`auto_arima`), **statsforecast** (very fast classical models over thousands of series), **sktime** (forecasting with a scikit-learn interface), **darts** (deep learning forecasters). Deep learning models such as N-BEATS and the Temporal Fusion Transformer exist and mostly matter with many related series and rich features; on a single monthly series they rarely beat SARIMA.
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026. The slowest step is the backtest with SARIMA, about 20 seconds.
 - **Companion files:** `companion/generate_riverstone_demand.py` (seed 20240) and `companion/generate_riverstone_sensors.py` (seed 20241; `--full` builds the 4.8-million-row version for Chapter 48). Run the chapter's code from `companion/ch40/`. Data specs: `planning/data/riverstone-demand.md` and `riverstone-sensors.md`.
-

The project: forecast Riverstone's monthly demand by product category

Goal: a forecast for 2026 by category, with intervals, a backtested comparison against seasonal naive, and a one-page note the production planner could use.

Option A: your own data. Any monthly or weekly series with at least three years of history: sales, tickets, web traffic, energy use.

Option B: Riverstone. The monthly and weekly demand files.

Steps:

1. **Check the data.** Find and fix duplicates and gaps; plot every series.
2. **Decompose** each category and describe its trend and seasonal factors in one sentence each.
3. **Baselines.** Compute naive, seasonal naive, and moving average WAPE for 2025.
4. **Fit** Holt–Winters, SARIMA, and Prophet for each category. Record the parameters.
5. **Backtest** every method over at least four folds and tabulate mean and worst WAPE. Choose a method per category and justify it.
6. **Forecast 2026** with the chosen method, with 80% intervals.
7. **Size safety stock** from the backtest error and show the plan for the first six months.

8. **Weekly ML model** for one category with lag features, evaluated recursively, and compare with the monthly result.
9. **Write the planner's note:** the forecast, the interval, what it would have got wrong in 2020–2021, and what to watch.

Stretch goals:

- Add a **dummy variable** for the 2020 shutdown months to SARIMA (`exog=`) and see whether the 2021 fold improves.
- Forecast **all four categories at once** with `statsforecast` and compare its `AutoARIMA` choices with yours.
- Build a **direct multi-horizon** ML model (one model per horizon, 1 to 12 months) and compare with the recursive approach.
- Run the sensor anomaly rules on the `--full` dataset for one more machine and tune the persistence window.

You've got it when...

- ☐ I can name trend, seasonality, cycle, and noise in a series and describe what a decomposition shows.
- ☐ I check for duplicates and gaps before modeling.
- ☐ I know what stationarity is, what differencing does, and how to read an ADF p-value.
- ☐ I always compute the seasonal naive baseline first.
- ☐ I can do one step of exponential smoothing by hand and explain α , β , and γ .
- ☐ I can read an ACF and PACF plot well enough to suggest SARIMA orders.
- ☐ I fit SARIMA on a log scale when seasonality grows, and I always show the prediction interval.
- ☐ I build lag features with `shift` before `rolling`, split by time, and evaluate recursively for multi-step forecasts.
- ☐ I backtest with rolling origins and report the mean and worst fold.
- ☐ I quote WAPE, explain why not MAPE, and use MASE to compare series.
- ☐ I can size safety stock from backtest error and explain the trade-off.
- ☐ I know why point z-scores miss slow drift, and I use a slow baseline with a persistence rule.

Recap

- A time series has **trend**, **seasonality**, **cycles**, and **noise**; **decomposition** separates them for understanding, not forecasting.
- **Stationarity** is what classical models need; **differencing** (plain and seasonal) and **logs** get you there; the **ADF test** checks.

- **Seasonal naive** is the baseline to beat, and it often isn't beaten.
- **Exponential smoothing** weights recent values more; **Holt-Winters** adds trend and seasonality, and can fail when parameters are chosen by in-sample fit.
- **SARIMA** combines autoregression, differencing, and moving-average terms at ordinary and seasonal lags; read **ACF/PACF**, fit on logs, show **intervals**.
- **Prophet** is a strong low-effort option; **ML with lag features** competes on weekly data and wins with external features, if you avoid leaks and evaluate **recursively**.
- **Backtest** with rolling origins; one test year lies.
- **WAPE** is the planner's metric; **MAPE** misleads on small months; **MASE** compares across series.
- A forecast becomes a **plan** with safety stock sized from backtest error.
- Sensor anomalies: **status first**, **fast baselines for spikes**, **slow robust baselines for drift**, and **persistence** before alerting.

Practice exercises

Code exercises run from `companion/ch40/` after the chapter's code (they use `kitchen`, `monthly`, `weekly`, `train`, `test`, `waape`, `backtest`, `methods`, `forecasts`, `readings`, `temp`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) A series has these last four months: 100, 120, 110, 130. With $\alpha = 0.5$ and a starting level of 100, compute the smoothed level after each month.
2. (*hand*) A forecast for three months was 90, 100, 110 against actuals 100, 100, 100. Compute MAPE and WAPE. Now swap: forecast 100, 100, 100 against actuals 90, 100, 110. Compute both again, and explain why one measure changed and the other didn't.
3. (*hand*) Monthly series, 12-month seasonality, trending upward. Which SARIMA orders would you try first, and what does each of the six numbers mean?
4. Which baseline suits each series: (a) daily website visits with a strong weekly pattern; (b) a slowly rising count of active customers with no seasonality; (c) a stable product with occasional promotions?

Core

5. Decompose the **Industrial** series. What are its October and April seasonal factors, and how do they compare with Kitchen's? What does that imply for which method to use?
6. Fit Holt-Winters to Kitchen with `damped_trend=True` and with the seasonal parameter fixed at `smoothing_seasonal=0.2`. Does either beat the default's 17.5% on 2025?
7. Fit SARIMA orders $(1, 1, 0)(0, 1, 1)_{12}$ and $(0, 1, 1)(0, 1, 1)_{12}$. Compare AIC and 2025 WAPE with the chapter's model. Is the simpler model as good?
8. Compute an 80% prediction interval from SARIMA for every month of 2025 and count how many actuals fell inside it. Is that what an 80% interval should give?

9. Backtest Prophet with the same five folds. Where does it land against SARIMA and seasonal naive on the mean and the worst fold?
10. Rebuild the weekly ML features **without** `shift(1)` before the rolling means (the leaky version) and report the one-week-ahead WAPE. How much better does the leak make it look?

Stretch

11. Add a 0/1 `exog` variable for March–September 2020 to the SARIMA model and re-run the backtest. Which folds change, and by how much?
12. Fit one `HistGradientBoostingRegressor` per horizon (1, 4, 13, and 26 weeks ahead) on the weekly data, each with lags that respect its horizon, and compare their WAPE with the recursive model at the same horizons.
13. Apply the temperature persistence rule with thresholds of 4 °C and 8 °C, and windows of 5 and 20 minutes. For each combination report the alert time and the number of false alerts.

Think about it

14. The sales head asks for "the forecast" as one number per month. What two things do you add, and how do you explain them in a sentence each?
15. Your SARIMA model scored 4% on last year and 16% on the backtest fold that included the 2020 shutdown. A colleague says the 16% "doesn't count because that was a pandemic". Respond.
16. The maintenance team turned off the temperature alerts because they fired too often. What three changes would you make before turning them back on?

Key terms

time series · trend · seasonality · cycle · noise (residual) · decomposition · multiplicative model · stationarity · differencing · seasonal differencing · log transform · augmented Dickey–Fuller test · naive forecast · seasonal naive · moving average · exponential smoothing · alpha · Holt's method · Holt–Winters · damped trend · autocorrelation (ACF) · partial autocorrelation (PACF) · ARIMA · autoregressive (AR) · moving average (MA) · SARIMA · seasonal order · prediction interval · Prophet · `auto_arma` · lag feature · rolling feature · recursive forecast · direct multi-horizon · backtesting · rolling origin · fold · MAPE · WAPE · MASE · bias · safety stock · horizon · point anomaly · drift · rolling z-score · robust baseline · persistence rule · forecast residual · status masking

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 44, Capstone**, can take demand forecasting as its end-to-end project, from question to planner's note.
- **Chapter 45, Business Metrics**, puts rupee values on forecast error, safety stock, and stock-outs.

- **Chapter 48, Big Data and Distributed Computing**, runs this chapter's anomaly rules on 12 machines and 40 weeks with PySpark, DuckDB, and Polars.
- **Chapter 50, Streaming and Real-Time Data**, turns the sensor rules into alerts that fire as readings arrive.
- **Chapter 52, Deploying and Monitoring Models**, schedules the forecast, monitors its WAPE, and retrains it.
- **Chapter 30, Experiments**, and **Chapter 31, Causal Inference**, cover what forecasting can't: what happens if we change the price.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers stationarity, ARIMA versus ML, backtesting, MAPE versus WAPE, and "how would you forecast demand for a new product with no history?"

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Month 1: $0.5 \times 100 + 0.5 \times 100 = 100$. Month 2: $0.5 \times 120 + 0.5 \times 100 = 110$. Month 3: $0.5 \times 110 + 0.5 \times 110 = 110$. Month 4: $0.5 \times 130 + 0.5 \times 110 = 120$. The level ends at 120 while the last actual is 130: smoothing always lags a rising series, by more when α is small.
2. First case: errors 10, 0, 10 against actuals of 100 each. $\text{MAPE} = (10\% + 0\% + 10\%) \div 3 = 6.7\%$; $\text{WAPE} = 20 \div 300 = 6.7\%$. Second case: the same errors of 10, 0, 10, but the actuals are 90, 100, 110. $\text{WAPE} = 20 \div 300 = 6.7\%$ again, unchanged. $\text{MAPE} = (10 \div 90 + 0 + 10 \div 110) \div 3 = (11.1\% + 0 + 9.1\%) \div 3 = 6.7\%$ as well here, because the two misses happen to balance; try actuals 50, 100, 150 with the same errors and MAPE becomes $(20\% + 0\% + 6.7\%) \div 3 = 8.9\%$ while WAPE stays 6.7%. MAPE moves whenever the *same* error lands on a smaller actual; WAPE only depends on total error against total volume.
3. Start with $(1, 1, 1)(0, 1, 1)_{12}$ or its simpler cousin $(0, 1, 1)(0, 1, 1)_{12}$. The six numbers: $p = 1$ autoregressive term (this month depends on last month), $d = 1$ ordinary difference (to remove the trend), $q = 1$ moving-average term (this month depends on last month's error); then at the seasonal lag of 12: $P = 0$ seasonal AR terms, $D = 1$ seasonal difference (to remove the yearly pattern), $Q = 1$ seasonal MA term. Fit on the log scale if the seasonal swing grows with the level.
4. (a) **Seasonal naive with a 7-day period**: the same weekday last week. (b) **Naive with drift** (last value plus the average change), since there's no season to copy and a plain naive would ignore the rise. (c) **Seasonal naive or a moving average on non-promotion weeks**, with promotions handled separately as an adjustment, because no baseline should learn from a spike it can't predict.
- 5.

```

industrial = (
    monthly[monthly["category"] == "Industrial"]
    .set_index("month")["units"]
    .astype(float)
    .asfreq("MS")
)
ind_parts = seasonal_decompose(industrial, model="multiplicative", period=12)
ind_index = ind_parts.seasonal[:12]
ind_index.index = ind_index.index.strftime("%b")
print("Industrial seasonal index:", ind_index[["Apr", "Oct", "Nov"]].round(3).to_dict())
print(
    "Kitchen, for comparison: ",
    seasonal_index[["Apr", "Oct", "Nov"]].round(3).to_dict(),
)
print(f"Industrial seasonal range: {ind_index.min():.3f} to {ind_index.max():.3f}")

```

```

Industrial seasonal index: {'Apr': 0.949, 'Oct': 1.097, 'Nov': 1.023}
Kitchen, for comparison:  {'Apr': 0.845, 'Oct': 1.434, 'Nov': 1.3}
Industrial seasonal range: 0.928 to 1.097

```

Industrial's seasonal factors barely move (a few percent either side of 1), against Kitchen's 0.85 to 1.43. With so little seasonality, a seasonal model has nothing to model and mostly fits noise, which is why seasonal naive beat SARIMA on this category in section 40.9. For a series like this, a simple smoothing or drift method, judged by backtest, is the sensible choice.

6.

```

damped = ExponentialSmoothing(
    train, trend="add", damped_trend=True, seasonal="mul", seasonal_periods=12
).fit()
fixed_gamma = ExponentialSmoothing(
    train, trend="add", seasonal="mul", seasonal_periods=12
).fit(smoothing_seasonal=0.2)
print(
    f"damped trend:          WAPE 2025 {wape(test, damped.forecast(12)):.1%}    "
    f"alpha {damped.params['smoothing_level']:.2f}    "
    f"gamma {damped.params['smoothing_seasonal']:.3f}"
)
print(
    f"gamma fixed at 0.2:  WAPE 2025 {wape(test, fixed_gamma.forecast(12)):.1%}    "
    f"alpha {fixed_gamma.params['smoothing_level']:.2f}"
)

```

```

damped trend:          WAPE 2025 16.4%    alpha 0.69    gamma 0.000
gamma fixed at 0.2:  WAPE 2025 18.7%    alpha 0.70

```

Damping the trend helps a little (16.4%); fixing γ at 0.2 makes it worse (18.7%), because the optimizer then pushes α higher still. Neither comes close to seasonal naive's 10%. On this series Holt–Winters has a structural problem, not a tuning problem: the level term chases the festive spike and overshoots. The lesson is that in-sample optimizers need supervision on short series, and that the backtest (section 40.7), not one year, decides.

7.

```

for order in [(1, 1, 1), (1, 1, 0), (0, 1, 1)]:
    fit = SARIMAX(np.log(train), order=order, seasonal_order=(0, 1, 1, 12)).fit(
        disp=False
    )
    fc = np.exp(fit.forecast(12))
    print(
        f"order {order}(0,1,1)12: AIC {fit.aic:7.1f} WAPE 2025 {wape(test, fc):.1%}"
    )

```

```

order (1, 1, 1)(0,1,1)12: AIC   -58.0   WAPE 2025  4.0%
order (1, 1, 0)(0,1,1)12: AIC   -54.1   WAPE 2025 15.8%
order (0, 1, 1)(0,1,1)12: AIC   -54.1   WAPE 2025 15.7%

```

Dropping the MA(1) term is a disaster: WAPE jumps from 4.0% to about 16%, and the AIC gets worse (-54 against -58). The term with the alarming standard error was doing real work; its coefficient sits at the edge of the allowed range (-1), which is what inflated the standard error. The lesson runs against the usual advice to drop insignificant terms: for forecasting, judge a term by AIC and by out-of-sample error, not by its p-value alone. Simpler is better only when it forecasts as well.

8.

```

interval_80 = np.exp(sarima.get_forecast(12).conf_int(alpha=0.2))
inside = (test.to_numpy() >= interval_80.iloc[:, 0].to_numpy()) & (
    test.to_numpy() <= interval_80.iloc[:, 1].to_numpy()
)
print(f"months inside the 80% interval: {inside.sum()} of 12")
width = (interval_80.iloc[:, 1] - interval_80.iloc[:, 0]) / test.to_numpy()
print(
    f"interval width as a share of actual: {width.iloc[0]:.0%} "
    f"in January, {width.iloc[-1]:.0%} in December"
)

```

```

months inside the 80% interval: 12 of 12
interval width as a share of actual: 31% in January, 51% in December

```

An 80% interval should contain about 10 of 12 months. More than that means the intervals are wider than they need to be (the model's own error estimate includes the 2020 shock); fewer would mean overconfidence. Note that the interval widens with the horizon: December's is far wider than January's, which is honest, since a twelve-month-ahead forecast knows less.

9.

```
def prophet_forecast(history, n):
    model = Prophet(
        yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
    )
    model.fit(pd.DataFrame({"ds": history.index, "y": history.to_numpy()}))
    future = pd.date_range(
        history.index[-1] + pd.DateOffset(months=1), periods=n, freq="MS"
    )
    return model.predict(pd.DataFrame({"ds": future}))["yhat"].to_numpy()

prophet_scores = backtest(kitchen, prophet_forecast)
print(
    "Prophet folds:",
    [f"{v:.1%}" for v in prophet_scores],
    f" mean {prophet_scores.mean():.1%}   worst {prophet_scores.max():.1%}",
)
```

```
Prophet folds: ['33.4%', '10.8%', '7.7%', '8.5%', '5.9%']   mean 13.3%   worst 33.4%
```

Prophet's mean, 13.3%, is the worst of the four methods, and its 2021 fold is a 33% miss, far worse than anyone else's. Prophet fits a piecewise-linear trend, and a trend fitted through the 2020 collapse extrapolates the collapse into 2021. Its good 2025 score hid this. That's the backtest doing its job, and a reminder that the convenience of a method that "handles outliers" doesn't mean it handles shocks.

10.

```
leaky = pd.DataFrame({"units": wk})
for lag in [1, 2, 3, 4, 8, 13, 26, 52]:
    leaky[f"lag_{lag}"] = wk.shift(lag)
leaky["mean_4"] = wk.rolling(4).mean() # includes the current week: a leak
leaky["mean_13"] = wk.rolling(13).mean()
leaky["week_of_year"] = leaky.index.isocalendar().week.to_numpy().astype(int)
leaky["year"] = leaky.index.year
leaky = leaky.dropna()
lk_train, lk_test = leaky.loc[:"2024-12-31"], leaky.loc["2025"]
leaky_model = HistGradientBoostingRegressor(
    max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
)
leaky_model.fit(lk_train[X_cols], lk_train["units"])
print(
    f"leaky one-week-ahead WAPE: "
    f"{wape(lk_test['units'], leaky_model.predict(lk_test[X_cols])):.1%}   "
    f"(honest version: 9.3%)"
)
```

```
leaky one-week-ahead WAPE: 9.2%   (honest version: 9.3%)
```

The leak barely shows here: 9.2% against 9.3%. The current week enters `mean_4` at only a quarter weight, and `lag_1` already carries most of what the model needs, so the stolen information adds little. That's a reminder from Chapter 36: leaks aren't always spectacular, and a small score change doesn't prove a feature is clean. The column is still wrong. In production it can't be computed until the week is over, so the model would crash or be fed a stale value. Check every rolling feature for `shift` before you check its score.

11.

```
shutdown = pd.Series(
    ((kitchen.index >= "2020-03-01") & (kitchen.index <= "2020-09-01")).astype(int),
    index=kitchen.index,
)

def sarima_with_dummy(history, n):
    exog = shutdown.loc[history.index]
    fit = SARIMAX(
        np.log(history), exog=exog, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)
    ).fit(dispatch=False)
    future_exog = np.zeros((n, 1))
    return np.exp(fit.forecast(n, exog=future_exog)).to_numpy()

dummy_scores = backtest(kitchen, sarima_with_dummy)
plain_scores = backtest(kitchen, methods["SARIMA"])
for i, (a, b) in enumerate(zip(plain_scores, dummy_scores)):
    print(f"fold {i + 1}: without dummy {a:.1%}    with dummy {b:.1%}")
print(f"mean: {plain_scores.mean():.1%} -> {dummy_scores.mean():.1%}")
```

```
fold 1: without dummy 16.6%    with dummy 18.2%
fold 2: without dummy 15.0%    with dummy 14.4%
fold 3: without dummy 10.8%    with dummy 9.5%
fold 4: without dummy 9.9%     with dummy 8.0%
fold 5: without dummy 4.0%     with dummy 4.8%
mean: 11.3% -> 11.0%
```

The dummy helps folds 2 to 4 (forecasting 2022–2024) by one to two points, and hurts fold 1, which forecasts 2021 from a history where the dummy has only recently switched off, so the model has no clean post-shock months to learn the recovery from. The mean improves slightly (11.3% to 11.0%). Telling a model about a known one-off is cheap and usually worth it; it isn't magic, and the fold right after the shock is hard for every method.

12.

```
def horizon_features(series, horizon):
    frame = pd.DataFrame({"units": series})
    for lag in [1, 2, 4, 8, 13, 26, 52]:
        frame[f"lag_{lag}"] = series.shift(
            lag + horizon - 1
        ) # the newest value known h weeks before
    frame["mean_13"] = series.shift(horizon).rolling(13).mean()
    frame["week_of_year"] = frame.index.isocalendar().week.to_numpy().astype(int)
    return frame.dropna()

print("horizon    direct model    recursive model")
for horizon in [1, 4, 13, 26]:
    hf = horizon_features(wk, horizon)
    cols = [c for c in hf.columns if c != "units"]
    h_train, h_test = hf.loc[:"2024-12-31"], hf.loc["2025"]
    direct = HistGradientBoostingRegressor(
        max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
    )
    direct.fit(h_train[cols], h_train["units"])
    direct_wape = wape(h_test["units"], direct.predict(h_test[cols]))
    rec_wape = wape(wk_test["units"].iloc[:horizon], recursive.iloc[:horizon])
    print(f"{horizon:>7}    {direct_wape:11.1%}    {rec_wape:15.1%}")
```

horizon	direct model	recursive model
1	9.0%	0.3%
4	9.6%	7.2%
13	11.3%	6.8%
26	10.9%	6.7%

A direct model for each horizon avoids the compounding of recursive forecasts, at the cost of training one model per horizon and using older lags. The recursive column here is measured only over the first h weeks of 2025 (one week at $h = 1$, which is why it reads 0.3%), so the two columns aren't on the same footing; a full comparison would backtest both over many origins. In practice teams use direct models for a handful of business horizons (next week, next month, next quarter) and skip the rest.

13.

```
print("threshold window    first alert    alert minutes outside the fault")
fault_window = (temp.index >= "2025-03-05 22:00") & (temp.index <= "2025-03-05 23:30")
for threshold in [4, 8]:
    for window in [5, 20]:
        rule = (excess > threshold).rolling(window).sum() == window
        hits = rule[rule].index
        first_alert = f"{hits.min():%a %H:%M}" if len(hits) else "never"
        false_alerts = int((rule & ~fault_window).sum())
        print(f"{threshold:>9}    {window:>6}    {first_alert:<11}    {false_alerts}")
```

threshold	window	first alert	alert minutes outside the fault
4	5	Mon 11:13	132
4	20	Mon 13:14	40
8	5	Wed 22:51	0
8	20	never	0

A 4 °C threshold fires on Monday morning, long before any fault, and keeps firing: 40 to 132 false minutes, because the machine's normal daily warm-up crosses that line. At 8 °C with a 5-minute window the rule fires at 22:51 with no false alerts; with a 20-minute window it never fires, because the

fault didn't stay 8 °C above baseline for 20 minutes. Lower or shorter is earlier and noisier; higher or longer is quieter and later, and can miss the fault entirely. There's no right setting in the abstract: it depends on how costly a missed fault is against how quickly the team stops trusting alerts, which is section 39.5's cost trade-off again, now applied to minutes and degrees. Whatever you choose, record it in the same way as a model card.

14. A range and a baseline. "The range is where the actual will land four times out of five, based on how wrong the model has been in past years; plan for the top of it if a stock-out costs more than extra inventory." "The baseline is what last year's number would have given us; when the model isn't clearly better than that, we say so and use last year's number."

15. It counts precisely because it was a pandemic. The backtest's job is to show how the method behaves across the range of conditions that have actually occurred, and shocks are part of that range; a method scored only on calm years will be trusted exactly when it's about to fail. What's fair is to report both numbers with their context: "typically about 4–10% in a normal year, and about 16% through a shock like 2020, which the model can't foresee". Then decide the safety stock from the second number, not the first.

16. (1) Mask by status, so stoppages and start-ups don't trigger alerts. **(2) Replace point z-scores with two rules:** a fast residual rule for spikes and a slow-baseline rule with a persistence window for drift, each tuned on a week of history and checked against the known faults. **(3) Agree the alert budget with the team** (say, no more than two false alerts a week per machine) and tune thresholds to it, then review the alerts together for a month before making them page anyone. An alert that isn't trusted is worse than no alert, because it hides the real one.